

רמזים · המהדר קורא את הקוד שלך

שבוע: 12

מטרה

שלוש דרגות. פתח את הבאה רק אחרי שניסית. אין TA בחדר, ולכן זה מה שיש במקומו — ובשבוע הבא, במבחן, אין גם את זה.

דרגה 1 · דחיפה

"המהדר מוציא ארבעים שגיאות ואני לא יודע מאיפה להתחיל." תקן את הראשונה בלבד והרץ שוב. שגיאות טיפוסים מפילות זו את זו: שגיאה אחת בטיפוס מוקדם מייצרת עשר שגיאות במקומות שקוראים לו. ארבעים הופכות לשלוש די מהר.

"Property 'title' does not exist on type 'unknown'". בדיוק. זה בדיוק מה שביקשת ממנו לומר. unknown פירושו "אף אחד עוד לא בדק" — צריך משהו שיבדוק, לפני הגישה.

"איפה מתחילים בכלל?". ts/types.ts. לא api.ts. כל השאר מחזיר משהו שמתואר שם, ומי שמתחיל מהרשת מבלה עשר דקות בהמצאת שמות שהוא אחר כך משנה.

"אין לי מושג אם interface או type". שאלה אחת: האם זו צורה של אובייקט, או משהו אחר? צורה — interface. איחוד, חיתוך או כינוי — type. איחוד אי אפשר לכתוב כ- interface בכלל.

"מה זה '[Symbol.iterator]' must have a 'NodeListOf<Element>'?". זו לא בעיה בקוד שלך. חסר "DOM.Iterable" ב- lib — והוא כבר שם ב- tsconfig.json שקיבלת. אם השגיאה מופיעה, כנראה יש tsconfig.json שני, או שהעורך פותח תיקייה אחרת.

"הרצתי tsc ולא קרה כלום". פתח את js/. אם יש שם api.js שנוצר עכשיו — קרה הכול. אם אין — noEmitOnError דלוק, ויש שגיאה. גלול למעלה בטרמינל.

"האם למחוק את js/api.js הישן?". לא בידיים. תן למהדר לדרוס אותו. אם הוא לא דרס — הוא לא הצליח לקמפל.

דרגה 2 · אסטרטגיה

הגבול, בשלוש שורות

זו כל חלק ג, מרוכזת:

```
let payload: unknown;
payload = await res.json();
return toSuggestions(payload);
```

שורה 1 — אף אחד עוד לא בדק. שורה 2 — any הפך ל- unknown, בהצהרה אחת. שורה 3 — הפונקציה שבדקת, ומחזירה טיפוס אמיתי.

השורה שאסור לכתוב, והיא זו שהמכונה תציע:

```
const payload = (await res.json()) as WikiPayload;
```

מהדר. בודק כלום.

as אינו המרה ואינו בדיקה. הוא אומר למהדר להפסיק לשאול, בדיוק בגבול שבו איש אחר לא שואל.

מבנה השומר, אם נתקעת עליו

```
function isRecord(value: unknown): value is Record<string, unknown> {  
  return typeof value === 'object' && value !== null && !Array.isArray(value);  
}
```

isRecord ראשון, תמיד. `typeof null === 'object'` ו- `typeof [] === 'object'` – בלי שתי הבדיקות האלה, השומר שלך יאשר `null` ויקרוס בשורה הבאה.

ואז, בגוף `isItem`, שורה אחת לכל שדה שבלעדיו הציור נשבר:

```
if (typeof value.id !== 'string' || value.id === '') return false;
```

המחרוזת הריקה היא לא פינה. `''` הוא `string` תקין לגמרי מבחינת המהדר, והוא `id` לא חוקי מבחינת היישום. הטיפוס לא יכול לדעת את זה; השומר כן.

catch תחת **strict**

```
} catch (error) {  
  if (error instanceof Error && error.name === 'AbortError') throw error;
```

`error` הוא `unknown`, וזה נכון: ב-JavaScript אפשר לזרוק מחרוזת.

מה שהמכונה תציע: `catch (error: any)`. זה מכבה את השורה בדיוק במקום שבו היא שווה משהו – בטיפול בשגיאות, שהוא הקוד שאף אחד לא מריץ בזמן הפיתוח.

הגנריקה של חלק ד, בשורה אחת

```
export function <T extends Element>(selector: string): T {
```

תרגום: `T` הוא סוג אלמנט כלשהו (`extends Element` = לא כל דבר), **מי שקורא בוחר איזה**, וברירת המחדל `Element` = שומרת על קריאות בלי ארגומנט טיפוס.

מה שהופך את זה למועיל ולא לקישוט: ההצהרה קורית **פעם אחת**, בפונקציה עם שם, במקום שאפשר למצוא. `as HTMLInputElement` בארבעים מקומות הוא אותה הצהרה, ארבעים פעם, ואי אפשר לחפש אותה.

הבדיקה שסוגרת את חלק ג

```
npx tsc -p tsconfig.json  
git diff js/api.js
```

אם בדיף יש **תנאי** שהשתנה – שינית התנהגות. סוגריים, רווחים והזחה הם בסדר; `if` שנעלם או `&&` שהפך ל- `||` הוא לא.

דרגה 3 • כמעט הפתרון

`ts/types.ts` – השלד המלא

```
export type ErrorKind = 'offline' | 'timeout' | 'status' | 'shape';
export type RequestStatus = 'idle' | 'loading' | 'done' | 'error';

export interface Item {
  id: string;
  title: string;
  number: number;
  category: string;
  created: string; // ISO 8601. NOT a Date – see js/items.js
}

export interface Suggestion {
  readonly title: string;
  readonly note: string;
  readonly url: string;
}

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null && !Array.isArray(value);
}

export function isItem(value: unknown): value is Item {
  if (!isRecord(value)) return false;
  if (typeof value.id !== 'string' || value.id === '') return false;
  if (typeof value.title !== 'string' || value.title === '') return false;
  if (typeof value.number !== 'number' || !Number.isFinite(value.number)) return false;
  if (typeof value.category !== 'string') return false;
  if (typeof value.created !== 'string') return false;
  return true;
}
```

התאם את `Item` לשדות שלך. אם ה- `items.js` שלך נקרא אחרת – אלה השמות שמשנים, לא אלה.

```
import type { ErrorKind, Suggestion } from './types.js';

// Their schema, not ours. Declared here, and NOT exported.
interface WikiPage {
  title?: unknown;
  extract?: unknown;
  index?: unknown;
}

export class ApiError extends Error {
  readonly kind: ErrorKind;
  readonly messageHe: string;
  constructor(kind: ErrorKind, messageHe: string, detail: string) { /* unchanged */ }
}

export async function searchTitles(
  query: string,
  { signal, retries = 1 }: { signal?: AbortSignal; retries?: number } = {},
): Promise<Suggestion[]> {
```

כל השאר בגוף הפונקציה נשאר מילה במילה.

```
export function {{CONTENT}}lt;T extends Element = Element>(selector: string, root:
ParentNode = document): T {
  const found = root.querySelector(selector);
  if (found === null) throw new Error(`dom: no element matches ${selector}`);
  return found as T;
}

export function valueOf(selector: string): string {
  return {{CONTENT}}lt;HTMLInputElement>(selector).value.trim();
}
```

ואז, ב- `js/events.js`, שורת ייבוא אחת ושימוש אחד. זה מספיק כדי שהחלק ייחשב: עוזר שנכתב ולא נקרא הוא קובץ, לא ארגון מחדש.

```
import { valueOf } from './dom.js';
// ...
apiQuery.addEventListener('input', () => scheduleSearch(valueOf('#api-query')));
```

אם נגמר הזמן

חלקים א, ב ו-ג הם הליבה, והם 72 מתוך 100. אם נשארו לך עשר דקות – סיים אותם, ותכתוב ב- `PROMPTS.md` את הביקורת (חלק ה, 12 נקודות) לפני שאתה מתחיל את חלק ד. הביקורת לוקחת חמש דקות ושווה יותר מהגנריקה, ובניגוד אליה אי אפשר להשלים אותה בבית מהזיכרון.